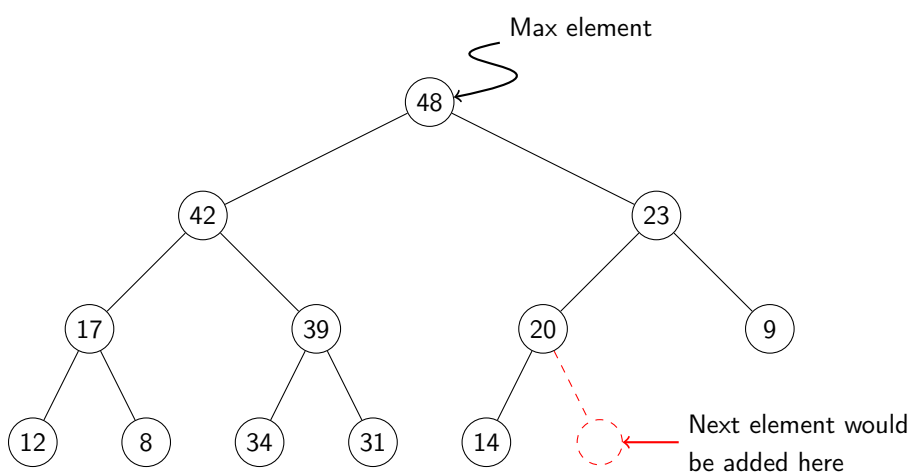
Motivation

Abstract Data Type **Maximum Priority Queue** is a collection of integers with the following operations.

`D.Init()`	Initializes the priority queue
`D.Insert(`x`)`	Inserts x into the Priority Queue.
int `D.ExtractMax()`	Extract the maximum element from the Priority Queue.

A heap is an implementation of Priority Queue that stores the values in a binary tree. The maximum value of the heap is located at the root of the tree and new elements are inserted (initially) as leaves of the tree.



Definition 1. A **max-heap** is a nearly complete binary tree that satisfies the heap property. The max-heap property is that for any node x, if c_l and c_r are the left and right children of x, then $x.\texttt{key} \geq c_l.\texttt{key}$ and $x.\texttt{key} \geq c_r.\texttt{key}$.

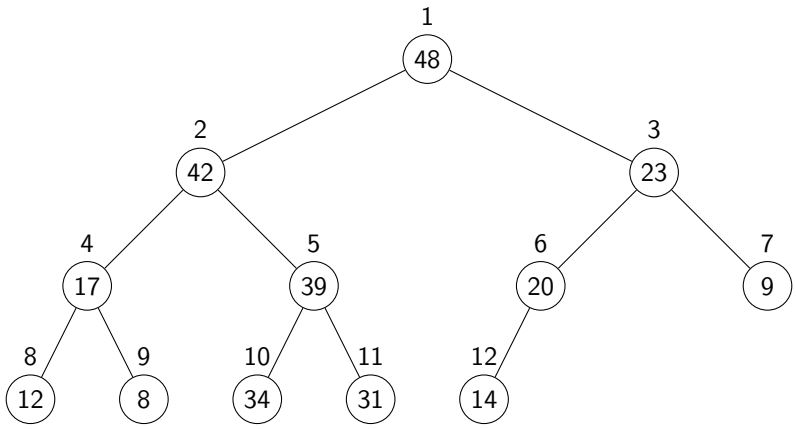
Definition 2. A **min-heap** is a nearly complete binary tree that satisfies the heap property. The min-heap property is that for any node x, if c_l and c_r are the left and right children of x, then $x.\texttt{key} \leq c_l.\texttt{key}$ and $x.\texttt{key} \leq c_r.\texttt{key}$.

Introduction and Methods

We implement a heap as a nearly complete binary tree; this means that the only "missing" leaves appear on the bottom level of the tree. More formally, a nearly complete binary tree of height h satisfies the following properties:

1. The ith level of the tree has 2^i nodes for $i = 0, 1, 2, \ldots, h-1$.
2. All nodes on the hth level of the tree are as far left as possible.

An example of a nearly complete binary search tree can be found below



Notice that we have numbered the nodes of the binary tree; using this ordering we observe that we can express the binary tree as the array (starting with index 1),

$$H = [48, 42, 23, 17, 39, 20, 9, 12, 8, 34, 31, 14]$$

and we observe the following about this particular labelling scheme:

1. The left child of every node has twice the index as the node
2. The right child is one more than the left child
3. The parent of node $H[i]$ is $H[\lfloor i/2 \rfloor]$

```
function Parent(i)
    return ⌊i/2⌋
end function
function Left(i)
    return 2i
end function
function Right(i)
    return 2i + 1
end function
```

We can also come with some convenient bounds on the height of a nearly complete binary tree. Let T be a nearly complete binary tree with n nodes and height h. The total number of nodes in the tree can be expressed as

$$n = \sum_{i=0}^{h} (\text{Number of nodes on level } i)$$

Furthermore, by Property 1 of a nearly complete binary tree, Number of nodes on level $i = 2^i$ for $i = 0, 1, 2, \ldots, h-1$ and since level h has at most 2^h nodes, so

$$n = \sum_{i=0}^{h} (\text{Number of nodes on level } i) \leq \sum_{i=0}^{h} 2^i = 2^{h+1} - 1.$$

We can rearrange this expression to find bounds on h,

$$h \geq \log_2(n+1) - 1$$

We can bound the height from above by a similar trick, in this case we observe that by Property 1,

$$n = \sum_{i=0}^{h} (\text{Number of nodes on level } i) \geq \sum_{i=0}^{h-1} 2^i = 2^h - 1.$$

We again rearrange this expression and solve for the height,

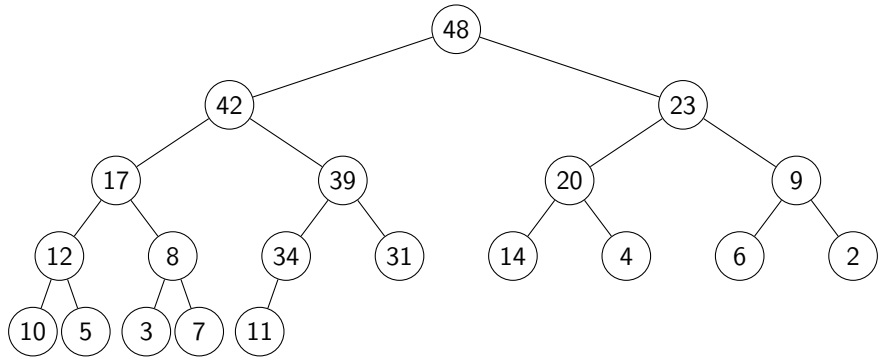
$$h \leq \log_2(n+1)$$

So,

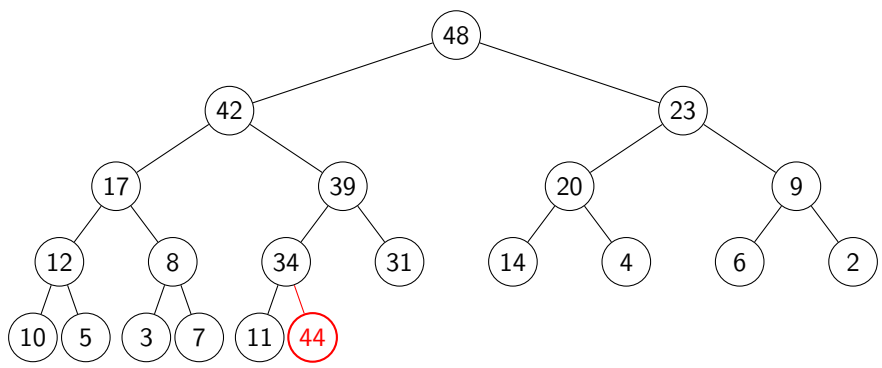
$$\log_2(n+1) - 1 \leq h \leq \log_2(n+1)$$

Thus, $h \in \Theta(\log_2(n))$.

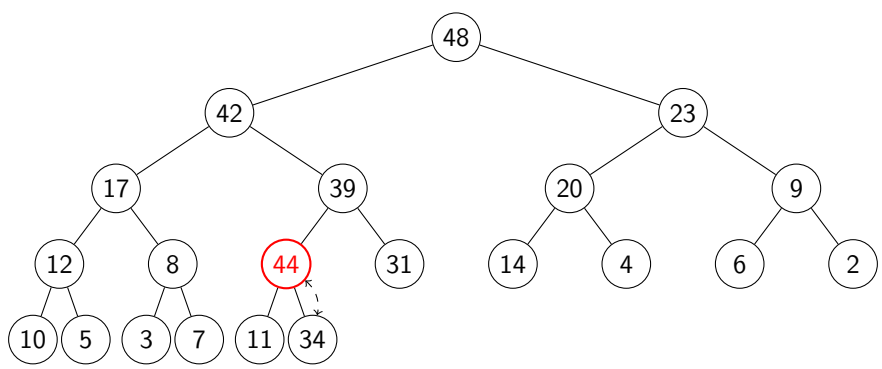
We shall now discuss how we can insert a node into a heap. Consider this following heap



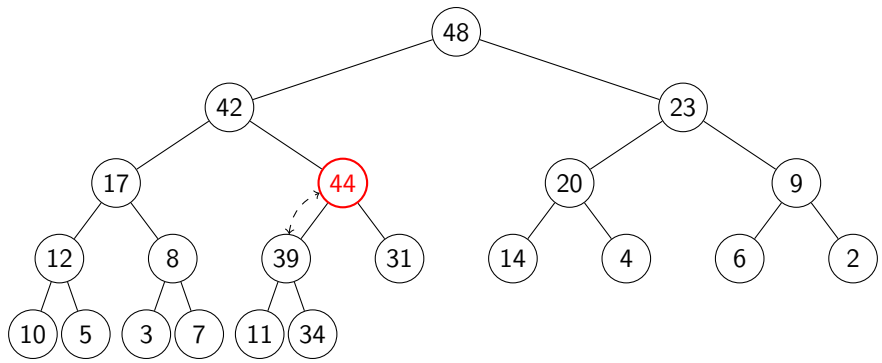
and suppose that we wanted to insert 44 into this heap. To do this, we insert 44 into the left most open location on the last child level,



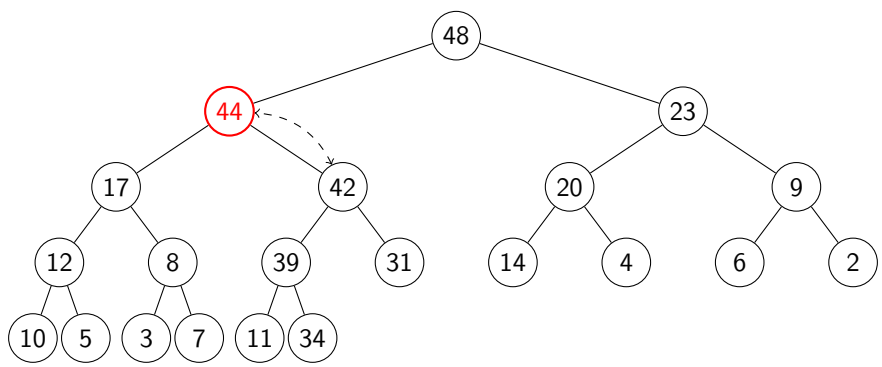
Notice that this insertion now violates the max-heap property since $34 < 44$; therefore, we must perform some operations to restore the heap property. One way to restore the max-heap property would be to switch the two nodes that are currently breaking the property. So,



Now, we can see that the entire subtree rooted at 44 obeys all of the max-heap properties since $44 > 34$ and we already knew that $44 > 11$ since $34 > 11$ and $44 > 34$! However, we have introduced another node that violates the heap property – namely, $44 < 39$. Therefore, we perform another swap, this time swapping 44 and 39.



Now observe that the entire subtree rooted at 44 is a valid max-heap, but $42 < 44$, therefore we must again perform a swap. This time we swap 44 and 42,



Now, as before, the subtree rooted at 44 is a valid max-heap *and* 44 < 48. Therefore, the entire data structure again obeys all of the max-heap properties. We implement heap insertion by making it follow the procedure we just performed:

Input: A heap A of size $size$ and an element K

Output: A heap A' of size $(size + 1)$ containing $A[1], A[2], \ldots, A[size], K$

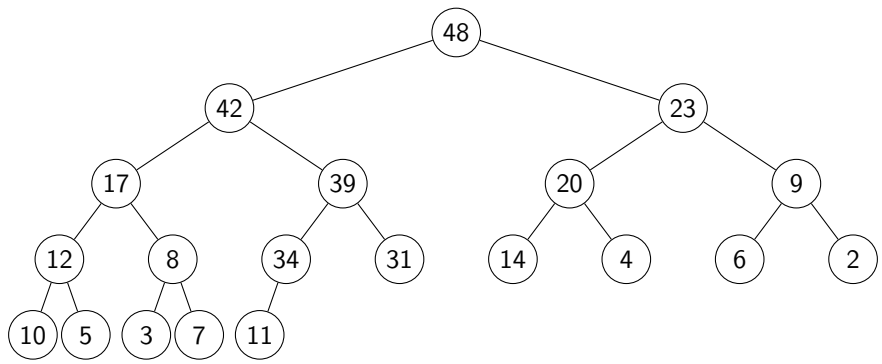
1: **function** `MaxHeapInsert`($A[]$,$size$,K)
2: $size = size + 1$
3: $i = size$
4: $A[i] = K$
5: **while** $(i > 1)$ **and** $(A[\texttt{Parent}(i)] < A[i])$ **do**
6: `Swap`($A[i]$,$A[\texttt{Parent}(i)]$)
7: $i = \texttt{Parent}(i)$
8: **end while**
9: **end function**

We shall only analyze the best and worst case running time of this method, the expected case requires slightly more sophisticated analysis than we are able to perform here. For those interested I recommend the analysis presented in the CLRS book!

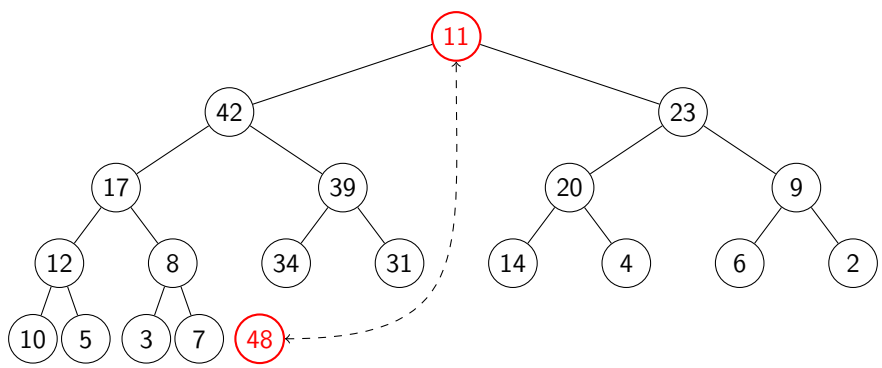
Best case The best case running time occurs when $K < A[\texttt{Parent}(size + 1)]$, in which case the while loop on line 5 is never entered and the algorithm takes $\Theta(1)$ time.

Worst case The worst case running time occurs when $K > A[1]$. In this case, we must swap all the way from a leaf to the root of the root of the tree. Therefore, the algorithm takes $\Theta(h)$ time in the worst case. However, recall that $h \in \Theta(n)$. Therefore, the worst case running time is $\Theta(\log(n))$!

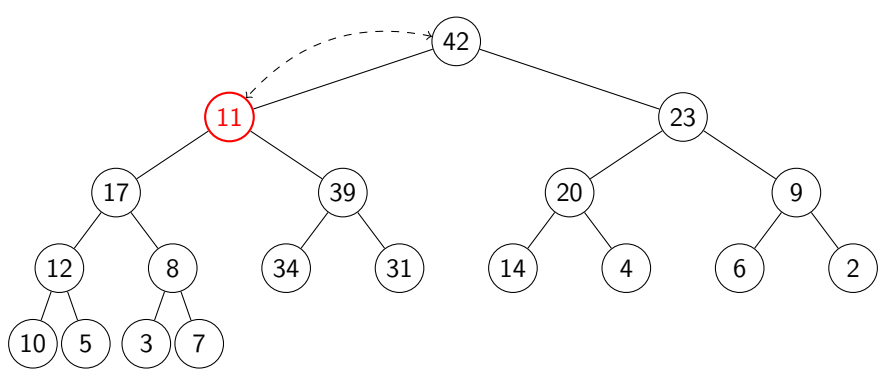
We shall now discuss how to extract the maximum element from a heap. Again consider the following heap:



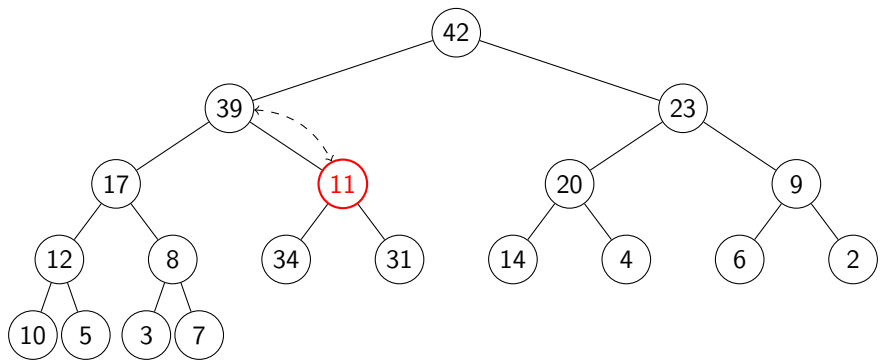
To extract the maximum element, we begin by swapping the root of the tree with the last element (the rightmost leaf on the lowest child level) and then remove the child. So the tree becomes



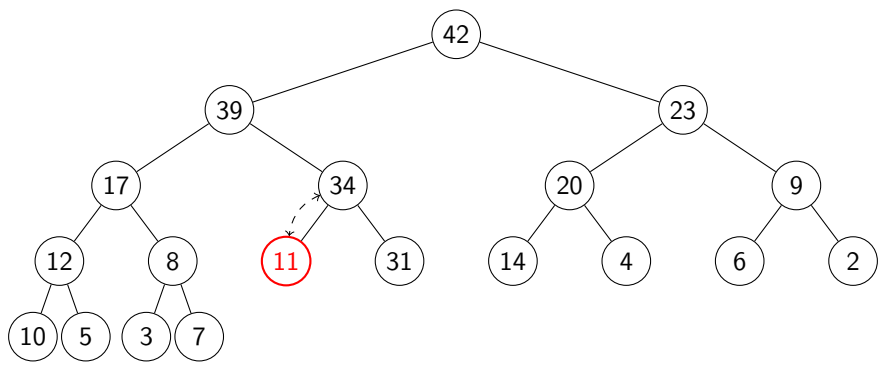
We must then restore the heap properties. Notice that the root is now less than BOTH of its children (i.e. $11 < 42$ and $11 < 23$). As we did with our insertion method, we shall swap the "problem" node with one the directly adjacent nodes. Furthermore, to ensure we restore the heap property we shall swap 11 with the *larger* of its two children. So, we swap 11 with 42 and the heap becomes



The heap properties remain broken, so we must perform another swap. Therefore, we swap 11 with the larger of its two children (39),



The heap properties remain broken, therefore we swap 11 with the larger of its two children (34),



And now the heap properties are restored! This is the process that we shall follow to extract the maximum element from the heap. The code for this is given below:

```
function HeapExtractMax(A[],size)
    if size < 1 then
        error("heap underflow")
    end if
    maxKey = A[1]
    A[1] = A[size]
    size = size − 1
    MaxHeapify(A[],size)
    return maxKey
end function
```

We also demonstrate two implementations of `MaxHeapify` that are functionally identical.

```
function MaxHeapify(A[],size)
    flagDone = false
    j = 1
    repeat
        L = Left(j)
        R = Right(j)
        ▷ Find index of largest element between A[j], A[L], A[R]
        largest = j
        if (L ≤ size) and A[L] > A[largest] then
            largest = L
        end if
        if (R ≤ size) and A[R] > A[largest] then
            largest = R
        end if
        if (j ≠ largest) then
            Swap(A[j],A[largest])
        else
            flagDone = true
        end if
        j = largest
    until flagDone
end function
```

```
function MaxHeapify(A[],size)
    flagDone = false
    j = 1
    repeat
        L = Left(j)
        R = Right(j)
        ▷ Find largest element of A[j], A[L], A[R]
        largest = arg max_{{j,L,R}} {A}
        if (j ≠ largest) then
            Swap(A[j],A[largest])
        else
            flagDone = true
        end if
        j = largest
    until flagDone
end function
```

As with `MaxHeapInsert` we shall only analyze the best and worst case running time of this algorithm. We first note that the running time of all of the statements in `HeapExtractMax` take constant time save for `MaxHeapify`. Therefore, we shall focus our analysis on `MaxHeapify`. And, as with insertion we shall again focus our analysis on the best and worst case.

Best case We first observe that when $n > 3$ it is impossible for the "new" root of the tree to be the larger than both of its children. This is true because the second largest element of a max heap must be a direct descendent of the root! Therefore, the best case is that the node that is moved to the root is larger than all of the children of the second largest element. In this case we perform two swaps so the best case running time is $\Theta(1)$.

Worst case In the worst case the new element is the minimum element of the max heap and must be swapped until it is again a leaf of the tree. In this case $\Theta(h)$ swaps occurs, therefore the worst case running time is $\Theta(\log(n))$.

The last important method we shall discuss is `MaxHeapIncreasKey`. This method allows us to increase the value of a node in the heap. We leave it to the reader to practice this method, but it functions extremely similarly to the insertion method. We "sift-up" from the value that we have changed.

```
function MaxHeapIncreaseKey(A[],size,i,K)
    if K < A[i] then
        error("new key is less than current key")
    end if
    A[i] = K
    while (i > 1) and (A[Parent(i)] < A[i]) do
        Swap(A[i],A[Parent(i)])
        i = Parent(i)
    end while
end function
```

The running time for this method is **identical** to the running time of `MaxHeapInsert`. Therefore, the worst case is $\Theta(\log(n))$ and the best case is $\Theta(1)$.

We pride ourselves on our endless ingenuity, so we shall use our new Heap data structure to solve a completely new problem: sorting. We shall introduce heap sort – this sorts an array by first turning the array into a heap and then iteratively extracts the maximum element and places it at the end of the array.

Input: Array B of n elements

Output: A permutation of B such that $B[1] \leq B[2] \leq \cdots \leq B[n]$

```
1: function HeapSort(B[],n)
2:     size = 0
3:     for i = 1 to n do
4:         MaxHeapInsert(A,size,B[i])
5:     end for
6:     for i = n downto 1 do
7:         B[i] = HeapExtractMax(A, size)
8:     end for
9: end function
```

Before we begin our analysis of this algorithm we mention that the running time of lines 4 and line 7 depend upon the size of the heap at that moment. We shall use $T_{in}(s)$ to represent the running time of `MaxHeapInsert` on a heap of size s and $T_{em}(s)$ to represent the running time of `HeapExtractMax` on a heap of size s. In our analysis we showed that the worst case running time of these methods is $\Theta(\log(s))$ and we will focus our analysis on the worst case. So we assume

$$T_{in}(s) = c\log(s) \qquad \text{and} \qquad T_{em}(s) = c\log(s).$$

If we let $T(n)$ be the running time of `HeapSort` on an array of size n, then

$$T(n) = \sum_{i=1}^{n} T_{in}(s) + \sum_{i=n}^{1} T_{ex}(s).$$

Here we have abused the summation notation – the second summation is designed to emulate the **downto** that appears on line 6. There are now two ways of proceeding. We shall demonstrate both of the methods here so that the reader can choose which they prefer.

Express s in terms of i We observe that every iteration of the for loop on lines 3-5 performs an insertion and that the heap is initially of size 0, therefore (for that loop) $s = i$. Furthermore, every iteration of the for loop on lines 6-8 performs an extraction, so $s = n - i + 1$ for the second loop. So,

$$T(n) = \sum_{i=1}^{n} T_{in}(i) + \sum_{i=1}^{n} T_{ex}(n - i + 1)$$

Substituting our assumed running times for our methods we have

$$T(n) = \sum_{i=1}^{n} c\log(i) + \sum_{i=1}^{n} c\log(n-i+1)$$

We proceed by bounding these summations above and below. Thus,

$$\begin{aligned} T(n) &= \sum_{i=1}^{n} c\log(i) + \sum_{i=1}^{n} c\log(n-i+1) \\ &\leq \sum_{i=1}^{n} c\log(n) + \sum_{i=1}^{n} c\log(n) \\ &\leq 2cn\log(n) \end{aligned}$$

and

$$\begin{aligned} T(n) &= \sum_{i=1}^{n} c\log(i) + \sum_{i=1}^{n} c\log(n-i+1) \\ &\geq \sum_{i=1}^{n} c\log(i) \\ &\geq \sum_{i=n/2+1}^{n} c\log\left(\frac{n}{2}\right) \\ &\geq \frac{cn}{2}\log\left(\frac{n}{2}\right) \end{aligned}$$

Therefore, $T(n) \in \Theta(n\log(n))$.

Express the sums in terms of s We can express the summations for each for loop in terms of the size of the heap. We observe that the initial size of the heap is 0 and after the first loop finishes execution the size is n. Furthermore, each iteration of that loop adds a single element to the heap. Similarly, the size before the second loops begins is n, the size after the loop finishes execution is 0, and a single element is removed each iteration. We comment that we typically use the size of the heap *after* insertion and *before* deletion. This allows us to start our summations at $s = 1$ and not $s = 0$. So,

$$T(n) = \sum_{s=1}^{n} c\log(s) + \sum_{s=n}^{1} c\log(s) = 2\sum_{s=1}^{n} c\log(s)$$

We now bound this summation to determine the complexity. So,

$$T(n) = 2\sum_{s=1}^{n} c\log(s) \leq 2cn\log(n)$$

and

$$T(n) = 2\sum_{s=1}^{n} c\log(s) \geq cn\log(n/2)$$

Therefore, $T(n) \in \Theta(n\log(n))$.

As a final comment on this problem we mention that the second method of expressing the summations in terms of the size of the data structure is the preferred method and is frequently easier than the alternative. Henceforth, we shall prefer this method but will make the concession of showing the first method when it is of interest to us.

Alternative Priority Queue Implementations

We began this unit by discussing the abstract data type of Priority Queue. We assumed this data structure had the following methods: **Operations:**

`D.Init()`	Initializes the priority queue
`D.Insert(`x`)`	Inserts x into the Priority Queue.
int `D.ExtractMax()`	Extract the maximum element from the Priority Queue.

And we have shown that our heap implementation had the following running times for the various methods **Heap:**

`D.Init()`	$\Theta(1)$ time
`D.Insert(`x`)`	$\Theta(\log(s))$ time
int `D.ExtractMax()`	$\Theta(\log(s))$ time

where s is the number of elements in the data structure.

The question now becomes: are there implementations of Priority Queue with superior running times? We shall discuss two alternative implementations.

Array implementation

Our first alternative is to implement the Priority Queue as an unsorted array. Initialization of this data structure is simple

```
function D.Init()
    length = 0
end function
```

This implementation allows for a *very* efficient insertion method,

```
function D.Insert(x)
    length = length + 1
    A[length] = x
end function
```

This method clearly takes constant time to execute! A marked improvement over our heap implementation! Unfortunately, there is no free lunch and we must pay a price *somewhere*. For this implementation we pay the price in the `ExtractMax` method. In order to extract the maximum element from an unsorted array we must always search the entire array. Therefore, the running time of `ExtractMax` on a Priority Queue of size s is $\Theta(s)$. This is a **much** worse running time than our heap implementation; this is a common occurrence with data structures, when you have two implementations you can rarely make one "objectively" superior. There are always trade-offs and costs to be paid for increased efficiency in some method.

```
function D.ExtractMax()
    if length < 1 then
        error("array underflow")
    end if
    for i = 1 to length − 1 do
        if A[i] > A[length] then
            Swap(A[i],A[length])
        end if
    end for
    x = A[length]
    length = length − 1
    return x
end function
```

Sorted array implementation

Our third implementation of Priority Queue will use a sorted array. Initialization is again simple,

```
function D.Init()
    length = 0
end function
```

This time our insertion method is more complicated. In particular, we shall implement `Insert` in the *exact* same manner as we implemented `SortedInsert` (the method used for `InsertionSort`). We have already shown that the worst and expected case running time of `SortedInsert` on an array of size s is $\Theta(s)$.

```
function D.Insert(x)
    length = length + 1
    A[length] = x
    k = length
    while (k > 1) and (A[k − 1] > A[k]) do
        Swap(A[k − 1], A[k])
        k = k − 1
    end while
end function
```

In this implementation we have a far inferior running time for insertion, but it is very easy to remove the maximum element from a sorted array, simply remove the last element and return it to the user. The running time of this method will be in $\Theta(1)$! So, as we saw with our unsorted array implementation, we can make improvements for one method (`ExtractMax`) while paying a price for other methods (`Insert`).

```
function D.ExtractMax()
    if length < 1 then
        error("array underflow")
    end if
    x = A[length]
    length = length − 1
    return x
end function
```

Priority queue sort

We now introduce a much more object-oriented implementation of `HeapSort`. This method generalizes the idea of the algorithm to *any* Priority Queue implementation!

Input: Array A of n elements

Output: A permutation of A such that $A[1] \leq A[2] \leq \cdots \leq A[n]$

```
function Sort(A[],n)
    D.Init()
    for i = 1 to n do
        D.Insert(A[i])
    end for
    for i = n downto 1 do
        A[i] = D.ExtractMax()
    end for
end function
```

We shall analyze the running time of this algorithm for each of our Priority Queue Implementations. Let $T(n)$ be the running time `Sort` on an array of size n. Regardless of the implementation we can borrow from our previous analysis of `HeapSort` and observe

- The size after the first insertion is $s = 1$.
- The size after the last insertion is $s = n$.
- The size before the first extraction is $s = n$.
- The size before the last extraction is $s = 1$.

so the running time of this algorithm can be expressed as summations over the size of the data structure as

$$T(n) = \sum_{s=1}^{n} T_{in}(s) + \sum_{s=1}^{n} T_{em}(s)$$

and the only difference between the implementations is the complexity of the methods used.

Heap For a heap implementation of Priority Queue we have that

$$T_{in}(s) = c\log(s) \qquad \text{and} \qquad T_{em}(s) = c\log(s)$$

so

$$T(n) = \sum_{s=1}^{n} c\log(s) + \sum_{s=n}^{1} c\log(s) = 2\sum_{s=1}^{n} c\log(s)$$

which we have shown is in $\Theta(n\log(n))$.

Unsorted array For an unsorted array implementation of Priority Queue we have that

$$T_{in}(s) = c \qquad \text{and} \qquad T_{em}(s) = cs$$

so

$$T(n) = \sum_{s=1}^{n} c + \sum_{s=n}^{1} cs = cn + c\frac{n(n+1)}{2}$$

So, $T(n) \in \Theta(n^2)$.

Sorted array For a sorted array implementation of Priority Queue we have that

$$T_{in}(s) = cs \qquad \text{and} \qquad T_{em}(s) = c$$

so

$$T(n) = \sum_{s=1}^{n} cs + \sum_{s=n}^{1} c = c\frac{n(n+1)}{2} + cn$$

So, $T(n) \in \Theta(n^2)$.

Therefore, `Sort` is most efficient when using the heap implementation of Priority Queue (in the worst case). We summarize our different implementations and their running times in the table below:

Operation	Heap	Array	Sorted Array
`Insert`	$\log(s)$	c	s
`ExtractMax`	$\log(s)$	s	c
`Sort`	$n\log(n)$	n^2	n^2

$s =$ number of elements in Priority Queue
$n =$number of elements in array being sorted.

Examples

Example 1. Determine the asymptotic complexity of the given algorithm:

```
1:  function Func(A[],n)
2:      D.Init()
3:      for i = 1 to n do
4:          for j = 1 to n do
5:              D.Insert(A[i] × A[j])
6:          end for
7:      end for
8:      x = 0
9:      for i = 1 to n do
10:         x = x + D.ExtractMax()
11:     end for
12:     return x
13: end function
```

Assuming that

`D.Insert`(x) is in $\Theta(s)$, where s is the number of elements in D

`D.ExtractMax()` is in $\Theta(1)$

This example perfectly demonstrates why analyzing the loops in terms of the size of the data structures is *much* easier than determining an expression for s in terms of the summation variables. For this problem we observe

- Every iteration of the for loops on lines 3-7 performs a single insertion.
- The size of the data structure after the first insertion is $s = 1$.
- The size of the data structure after the last insertion is $s = n^2$.
- The size of the data structure before the first extraction is $s = n^2$.
- The size of the data structure before the final extraction is $s = n^2 - n + 1 \approx n^2 - n$

So,

$$T(n) = \sum_{s=1}^{n^2} T_{in}(s) + \sum_{s=n^2}^{n^2-n} T_{em}(s)$$

Substituting the complexities of the methods we have

$$T(n) = \sum_{s=1}^{n^2} cs + \sum_{s=n^2-n}^{n^2} c = c\frac{n^2(n^2+1)}{2} + c(n+1)$$

Therefore, $T(n) \in \Theta(n^4)$.

For the sake of completeness we shall demonstrate how one could analyze this problem by expressing the size in terms of the summation indices. We first observe that every iteration of the inner loop increases the number of elements in the data structure by 1 and every iteration of the outer for loop increases the size of the data structure by n. Finally, we notice that the size on the first iteration (when $i = 1$ and $j = 1$ is 1; therefore,

$$s = j + (i - 1)n.$$

As we said above, when we increment j by 1, the size increases by 1 and when we increment i by 1 the size increases by n. Now, we must also compute the size of the data structure during the second loop. We observe that the initial size is n^2 and that it is reduce by 1 each iteration, so

$$s = n^2 - i$$

Therefore,

$$T(n) = \sum_{i=1}^{n} \sum_{j=1}^{n} T_{in}(j + (i - 1)n) + \sum_{i=1}^{n} T_{em}(n^2 - i)$$

Substituting the complexities of the methods we have

$$T(n) = \sum_{i=1}^{n} \sum_{j=1}^{n} c(j + (i - 1)n) + \sum_{i=1}^{n} c = \sum_{i=1}^{n} \sum_{j=1}^{n} c(j + (i - 1)n) + cn$$

We now must analyze the two summations correspond to the for loops on lines 3-7:

$$\begin{aligned}
T(n) &= \sum_{i=1}^{n} \sum_{j=1}^{n} c(j + (i - 1)n) + cn \\
&= \sum_{i=1}^{n} \left[c\frac{n(n+1)}{2} + (i - 1)n^2 \right] + cn^2 \\
&= cn\frac{n(n+1)}{2} + cn^2 \sum_{i=1}^{n} (i - 1) + cn^2 \\
&= \frac{cn^2(n+1)}{2} + cn^2 \cdot \frac{n(n-1)}{2} + cn^2
\end{aligned}$$

Therefore, $T(n) \in \Theta(n^4)$.
Yikes. That was unpleasant. 2/10 do not recommend.

Example 2. Determine the asymptotic complexity of the given algorithm:

```
function Func(A[],n)
    D.Init()
    for i = 1 to n do
        for j = 1 to n do
            D.Insert(A[i] × A[j])
        end for
    end for
    x = 0
    for i = 1 to n do
        x = x + D.ExtractMax()
    end for
    return x
end function
```

`D.Insert`(x) is in $\Theta(1)$

`D.ExtractMax`() is in $\Theta(s)$, where s is the number of elements in D

The code for this algorithm is identical to the previous example, therefore

$$T(n) = \sum_{s=1}^{n^2} T_{in}(s) + \sum_{s=n^2}^{n^2-n} T_{em}(s).$$

Substituting the complexities we have

$$T(n) = \sum_{s=1}^{n^2} c + \sum_{s=n^2-n}^{n^2} cs = cn^2 + c \sum_{s=n^2-n}^{n^2} cs$$

To analyze these summations we shall bound them above and below (we could get a closed form expression but feel that bounding is cleaner). So,

$$\begin{aligned} T(n) &= cn^2 + c \sum_{s=n^2-n}^{n^2} cs \\ &\leq cn^2 + c \sum_{s=n^2-n}^{n^2} cn^2 \\ &\leq cn^2 + c(n^2 - (n^2 - n) + 1)cn^2 \\ &\leq cn^2 + c(n+1)n^2 \end{aligned}$$

and

$$
\begin{aligned}
T(n) &= cn^2 + c \sum_{s=n^2-n}^{n^2} cs \\
&\geq c \sum_{s=n^2-n}^{n^2} c(n^2 - n) \\
&\geq c(n^2 - (n^2 - n) + 1)(n^2 - n) \\
&\geq cn(n^2 - n)
\end{aligned}
$$

Therefore, $T(n) \in \Theta(n^3)$.

Example 3. Determine the asymptotic complexity of the given algorithm:

```
function Func(A[],n)
    D.Init()
    for i = 1 to n do
        for j = 1 to n do
            D.Insert(A[i] × A[j])
        end for
    end for
    x = 0
    for i = 1 to n do
        x = x + D.ExtractMax()
    end for
    return x
end function
```

`D.Insert`(x) is in $\Theta(\log(s))$, where s is the number of elements in D

`D.ExtractMax`() is in $\Theta(\log(s))$, where s is the number of elements in D

The code for this algorithm is identical to the previous example, therefore

$$T(n) = \sum_{s=1}^{n^2} T_{in}(s) + \sum_{s=n^2}^{n^2-n} T_{em}(s).$$

Substituting the complexities we have

$$T(n) = \sum_{s=1}^{n^2} c\log(s) + \sum_{s=n^2-n}^{n^2} c\log(s)$$

Since we have logarithmic summations we must bound them to determine the complexity. So,

$$\begin{aligned}
T(n) &= \sum_{s=1}^{n^2} c\log(s) + \sum_{s=n^2-n}^{n^2} c\log(s) \\
&\leq \sum_{s=1}^{n^2} c\log(n^2) + \sum_{s=n^2-n}^{n^2} c\log(n^2) \\
&\leq cn^2\log(n^2) + c(n^2 - (n^2-n) + 1)\log(n^2) \\
&\leq 2cn^2\log(n) + 2c(n+1)\log(n)
\end{aligned}$$

and

$$
\begin{aligned}
T(n) &= \sum_{s=1}^{n^2} c\log(s) + \sum_{s=n^2-n}^{n^2} c\log(s) \\
&\geq \sum_{s=1}^{n^2} c\log(s) \\
&\geq \sum_{s=\frac{n^2}{2}+1}^{n^2} c\log(n) \\
&\geq \frac{cn^2}{2}\log(n)
\end{aligned}
$$

Therefore, $T(n) \in \Theta(n^2 \log(n))$.